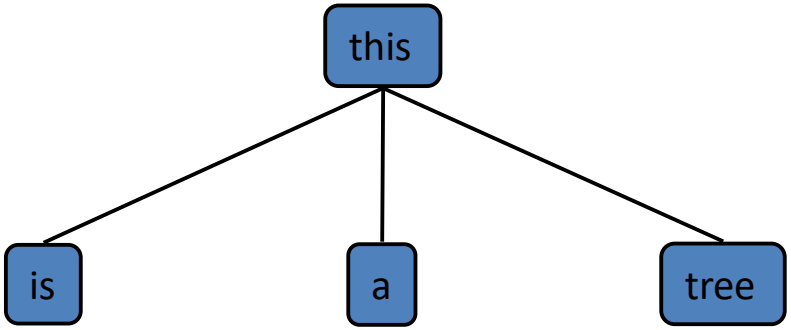
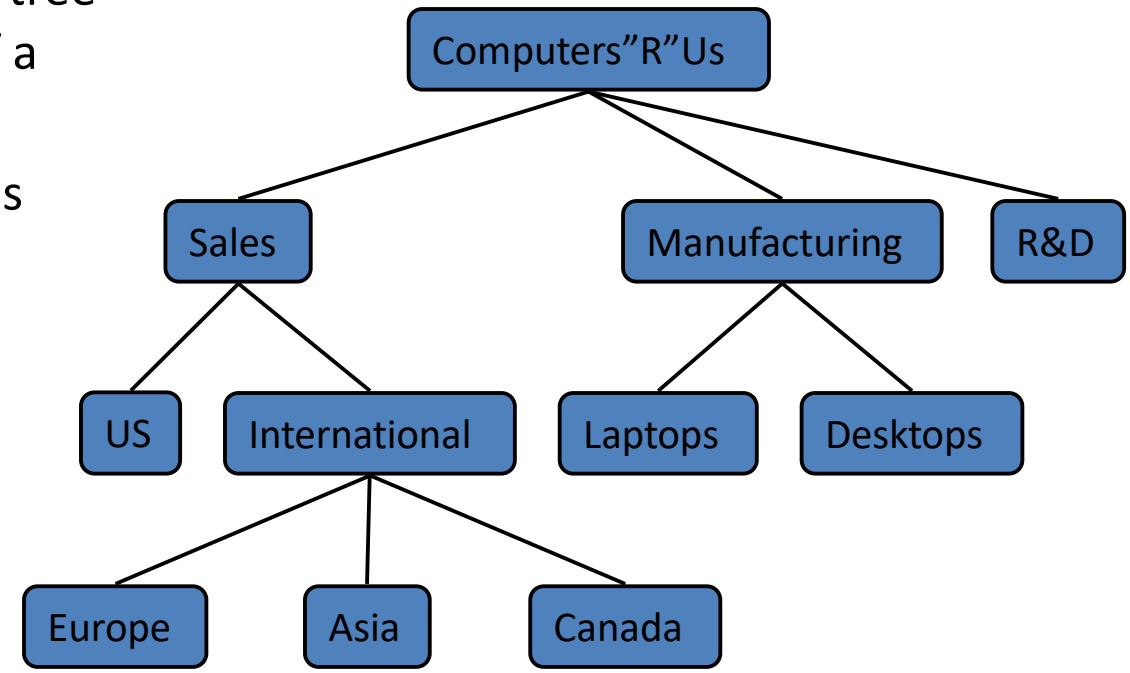
Trees



What is a Tree

- In computer science, a tree is an abstract model of a hierarchical structure
- A tree consists of nodes with a parent-child relationship
- Applications:
  - Organization charts
  - File systems
  - Programming environments



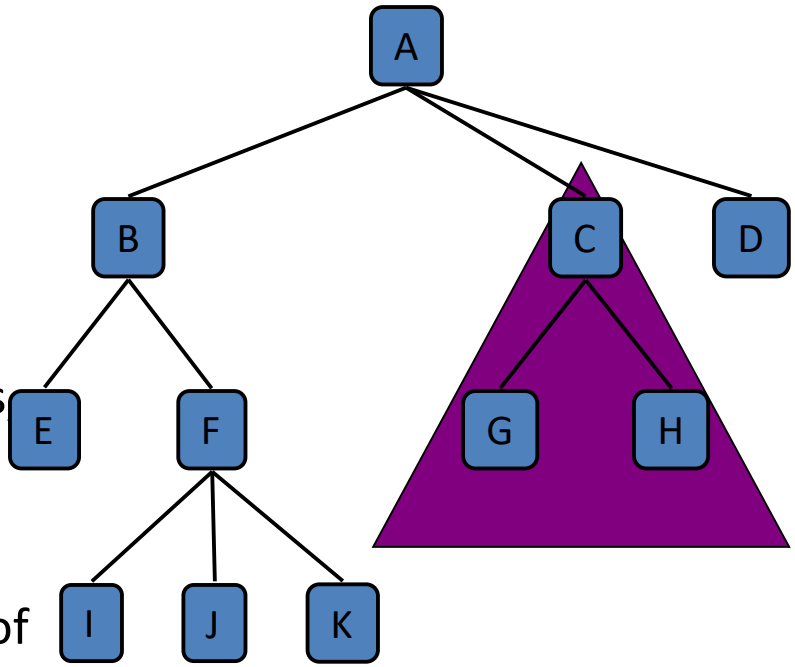
What is a Tree

- A connected acyclic graph is a tree.
- A tree *T* is a set of nodes in a parent-child relationship with the following properties:
  - *T* has a special node *r*, called the root of *T*, with no parent node
  - Each node *v* of *T*, different from *r*, has a unique parent node *u*
- A tree cannot be empty, since it must have at least one node – the root.

Tree Terminology

- Root: node without parent (A)
- Internal node: node with at least one child (A, B, C, F)
- External node (Leaf): node without children (E, I, J, K, G, H, D)
- Ancestors of a node: parent, grandparent, grand-grandparent, etc.
- Descendants of a node: child, grandchild, grand-grandchild, etc.
- Depth of a node: number of ancestors excluding the node itself
- Height of a tree: maximum depth of any node (3)
- Siblings: two nodes that are children of the same parent

- Subtree: tree consisting of a node and its descendants



subtree

Depth and Height

- The depth of a node *v* can be recursively defined as follows
  - If v is the root, then the depth of v is 0.
  - Otherwise, the depth of v is one plus the depth of the parent of v

```
Algorithm depth(T, v)
    if T.isRoot(v) then
      return 0
    else
      return 1 + depth(T, T.parent(v))
```

Running time: $O(1 + d_v)$, d_v is depth of v in T

In worst case $O(n)$, n is the number of nodes in T

Depth and Height

- The height of a node *v* can be recursively defined as follows
  - If v is a leaf node, then the height of v is 0.
  - Otherwise, the height of v is one plus the maximum height of a child of v

The height of a tree *T* is the height of the root of *T*

The height of a tree *T* is equal to the maximum depth of a leaf node of *T*

Depth and Height

```
Algorithm height1(T)
    h = 0
    for each v εT.positions(v) do
      if T.isExternal(v) then
        h = max(h, depth(T, v))
    return h
```

Running time: $O(n + \Sigma_{v \varepsilon E} (1 + d_v))$, where d_v is depth of v in T, E is the set of leaves in T

In worst case $O(n^2)$, n is the number of nodes in T

Depth and Height

```
Algorithm height2(T, v)
    if T.isExternal(v) then
        return 0
    else
        h = 0
        for each w ε T.children(v) do
            h = max(h, height2(T, w))
        return 1 + h
```

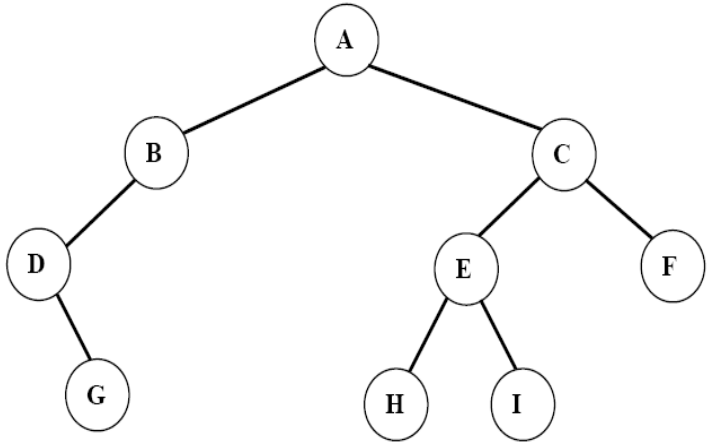
Running time: $O(\Sigma_{v \varepsilon T} (1 + c_v))$, where c_v is the number of children of v in T

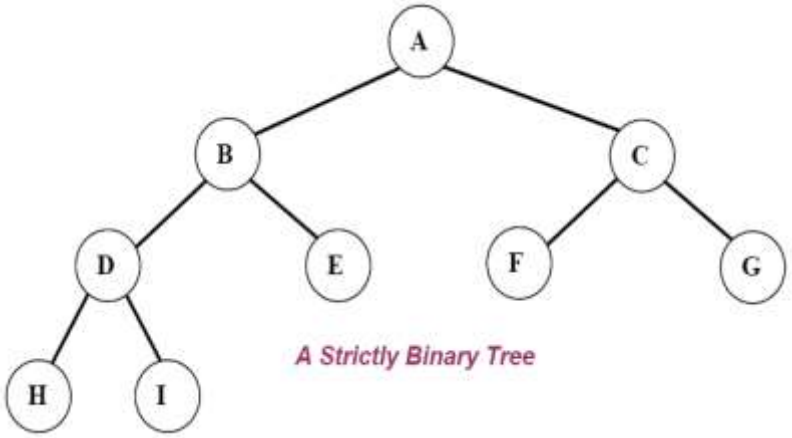
Since $\Sigma_{v \varepsilon T} c_{v =} n$ - 1, we have $O(\Sigma_{v \varepsilon T} (1 + c_v)) = O(n)$.

- The height of tree T is obtained by calling height2(T, r).

Ordered Trees

- A tree is ordered if there is a linear ordering defined for each child of each node.
- A binary tree is an ordered tree in which every node has at most two children.
- If each node of a tree has either zero or two children, the tree is called a proper (strictly) binary tree.





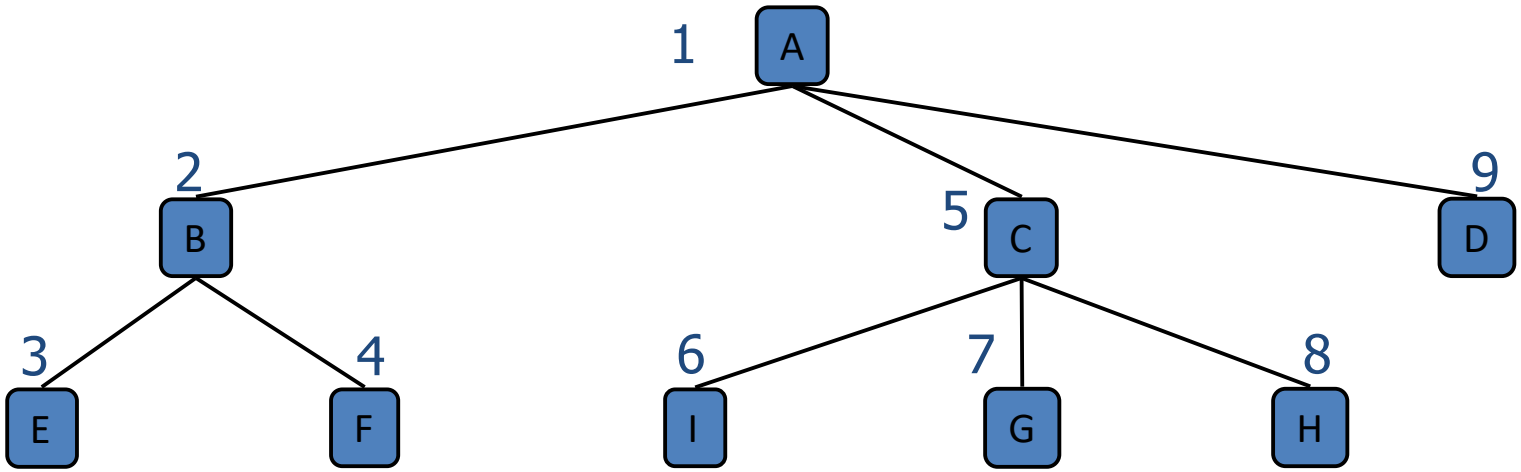
Traversal of Trees

- A traversal of a tree T is a systematic way of visiting all the nodes of T
- Traversing a tree involves visiting the root and traversing its subtrees
- There are the following traversal methods:
  - Preorder Traversal
  - Postorder Traversal
  - Inorder Traversal (of a binary tree)

Preorder Traversal

- In a preorder traversal, a node is visited before its descendants
- If a tree is ordered, then the subtrees are traversed according to the order of the children

Algorithm *preOrder*(*v*)
    visit(*v*)
    for each child *w* of *v*
        preorder (*w*)

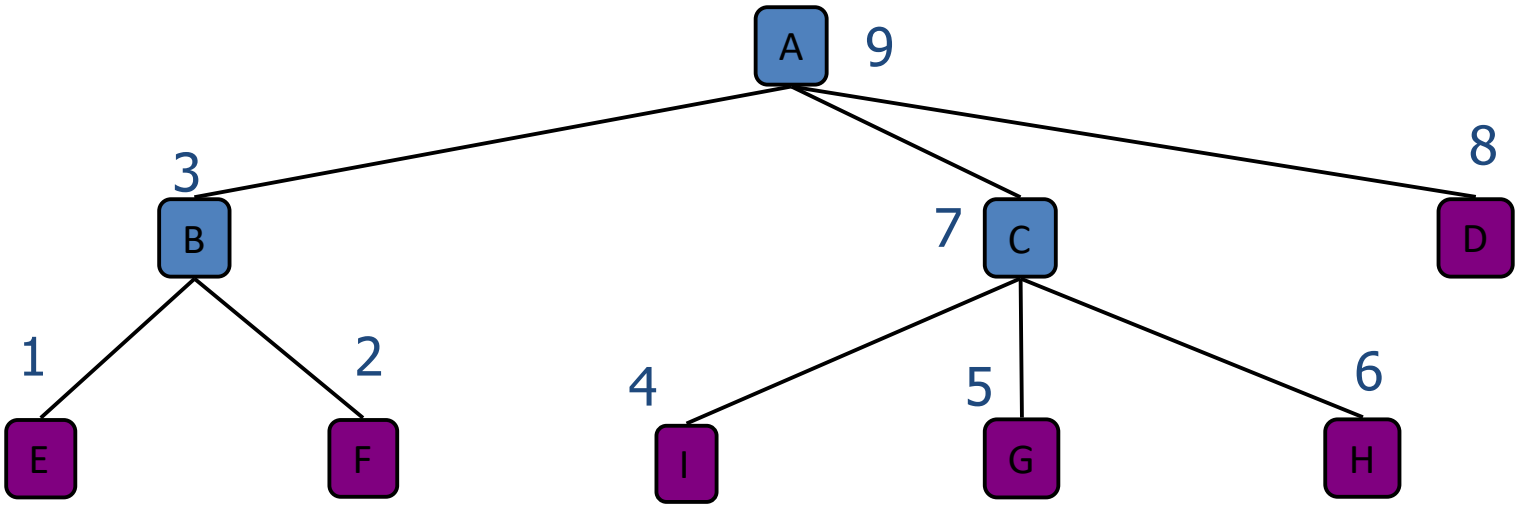


Preorder: *ABEFCIGHD*

Postorder Traversal

- In a postorder traversal, a node is visited after its descendants

Algorithm *postOrder*(*v*)
  for each child *w* of *v*
    postOrder (*w*)
  visit(*v*)

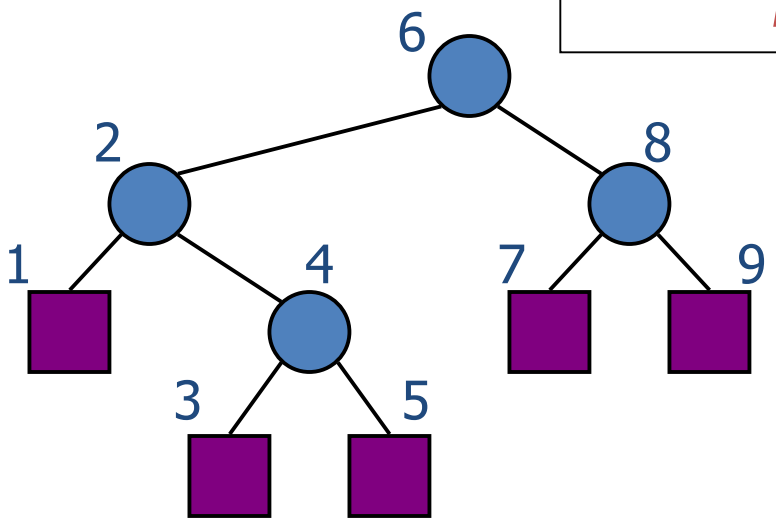


Postorder*: EFBIGHCDA*

Inorder Traversal

- In an inorder traversal a node is visited after its left subtree and before its right subtree

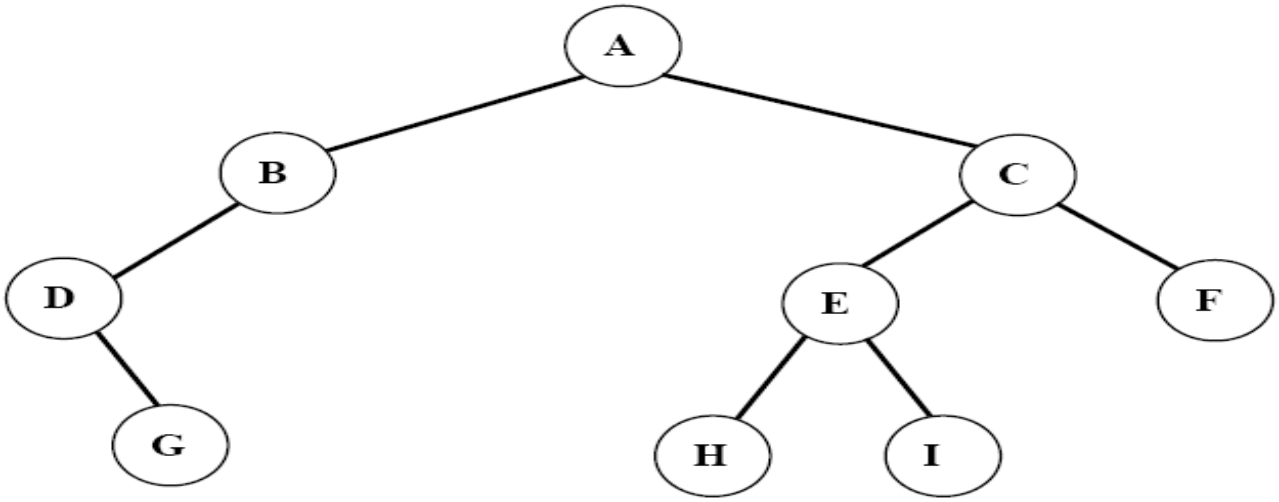
```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```



Inorder Traversal

Traversing a binary tree in *inorder*

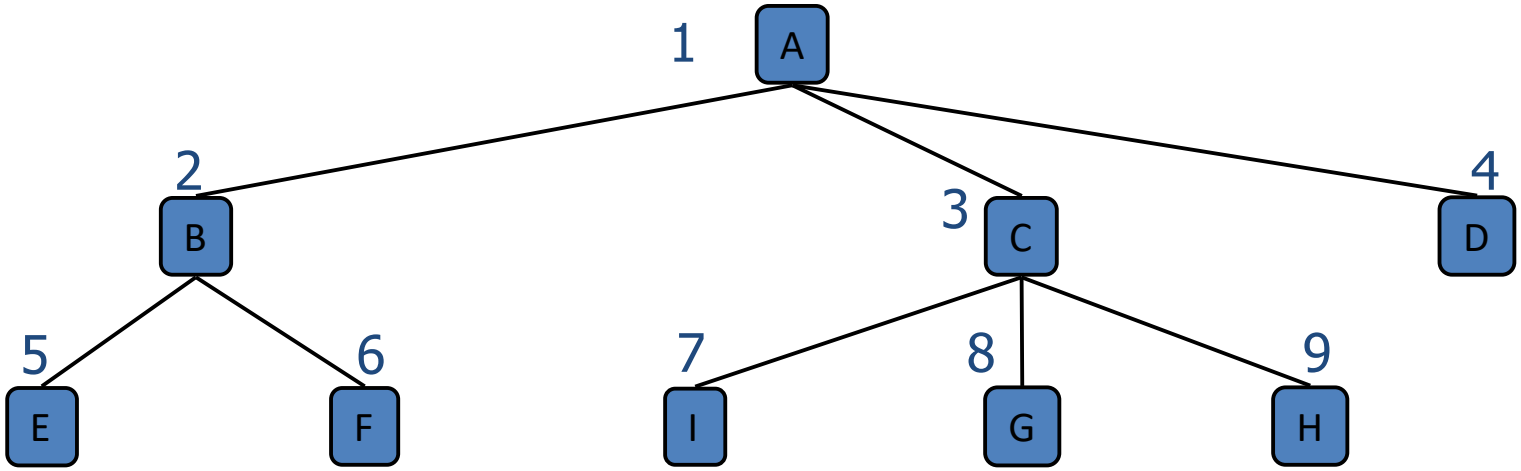
1. Traverse the ***left subtree*** in inorder.
2. Visit the ***root***.
3. Traverse the ***right subtree*** in inorder.



Inorder*: DGBAHEICF*

Level Order Traversal

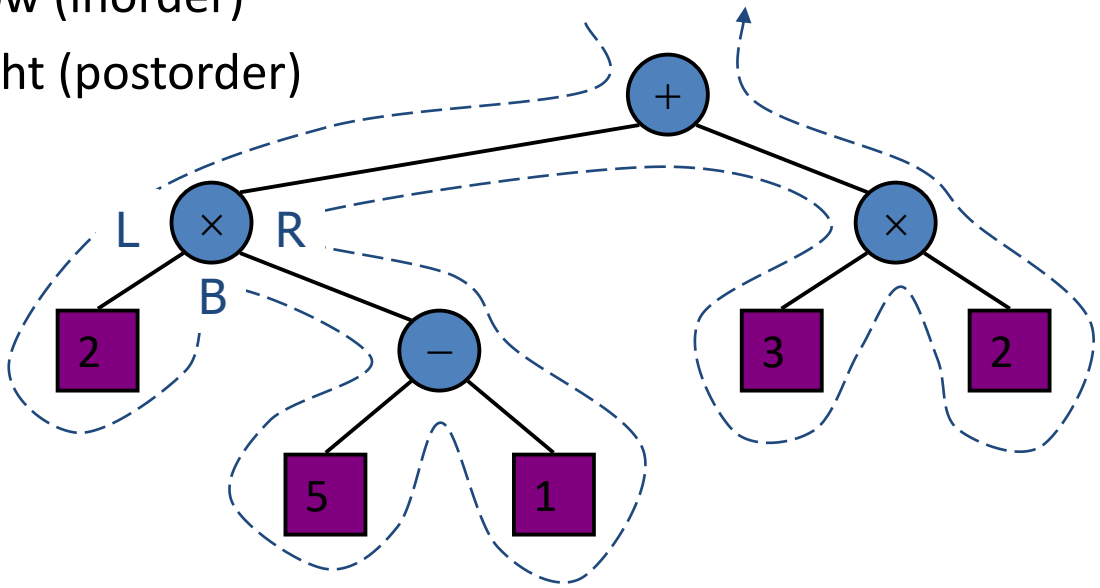
- In a level order traversal, every node on a level is visited before going to a lower level



Level order*: ABCDEFIGH*

Euler Tour Traversal

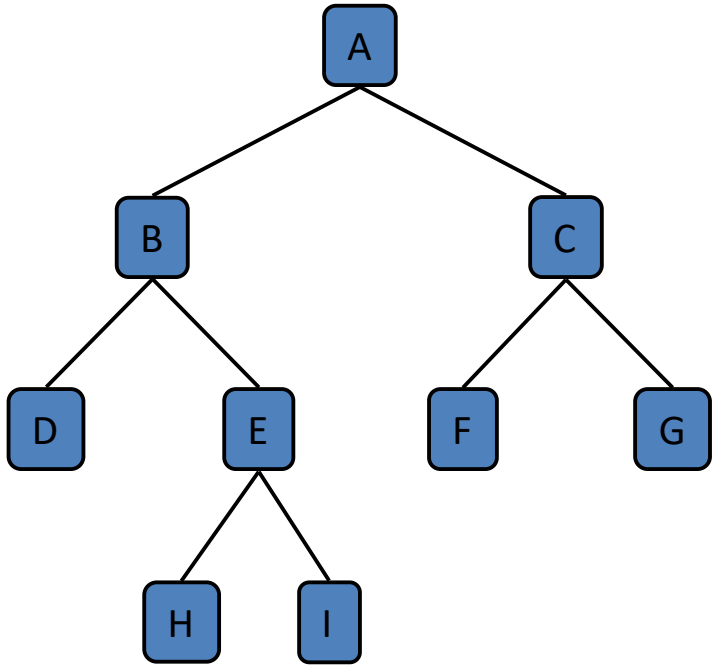
- Generic traversal of a binary tree
- Includes a special cases the preorder, postorder and inorder traversals
- Walk around the tree and visit each node three times:
  - on the left (preorder)
  - from below (inorder)
  - on the right (postorder)



(Proper) Binary Tree

- A (proper) binary tree is a tree with the following properties:
  - Each internal node has two children
  - The children of a node are an ordered pair
- We call the children of an internal node left child and right child
- Alternative recursive definition: a (proper) binary tree is either
  - a tree consisting of a single node, or
  - a tree whose root has an ordered pair of children, each of which is a binary tree

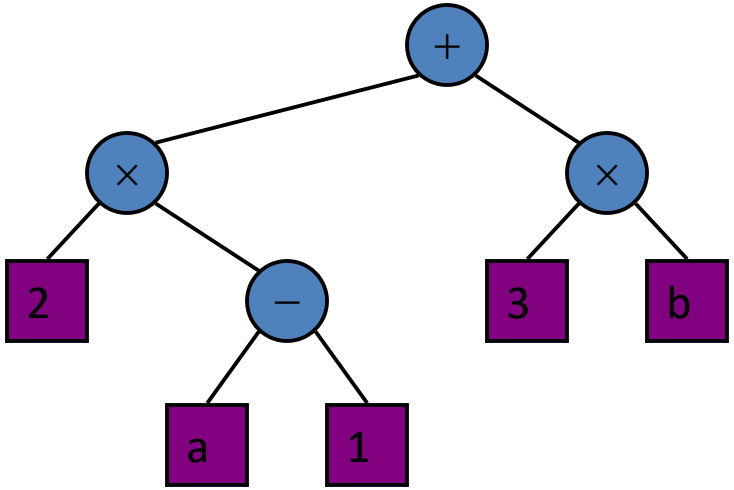
- Applications:
  - arithmetic expressions
  - decision processes
  - searching



Arithmetic Expression Tree

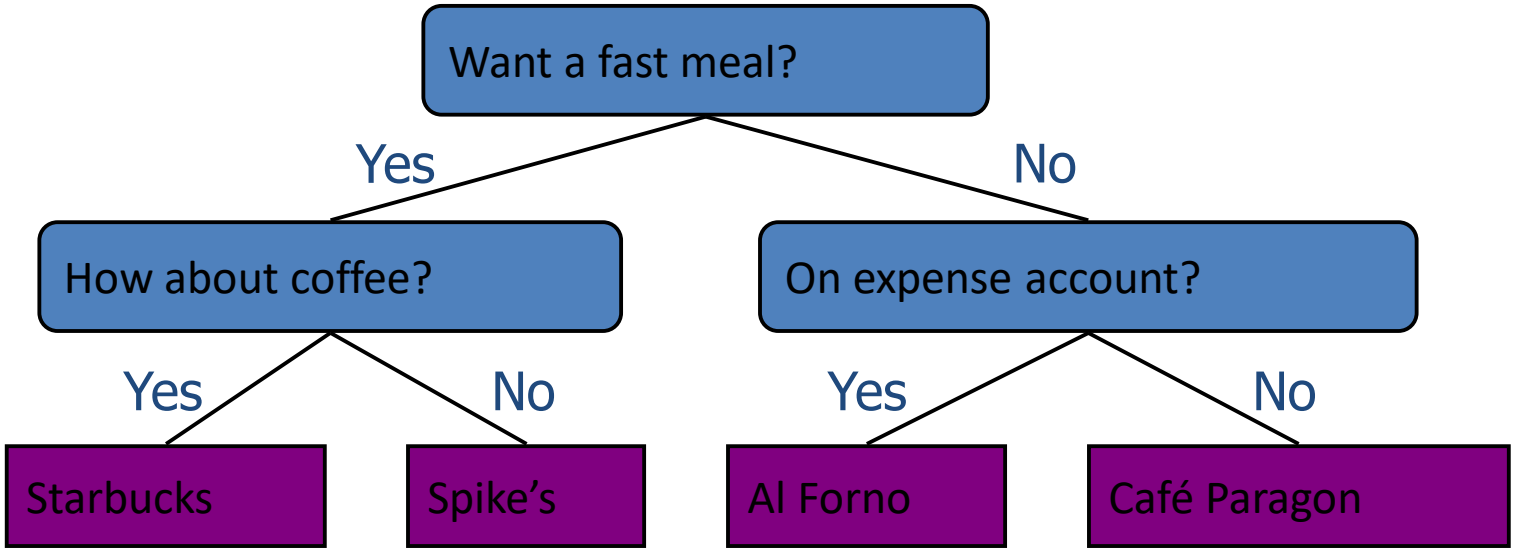
- Binary tree associated with an arithmetic expression
  - internal nodes: operators
  - external nodes: operands
- Example: arithmetic expression tree for the expression

  $(2 \times (a - 1) + (3 \times b))$



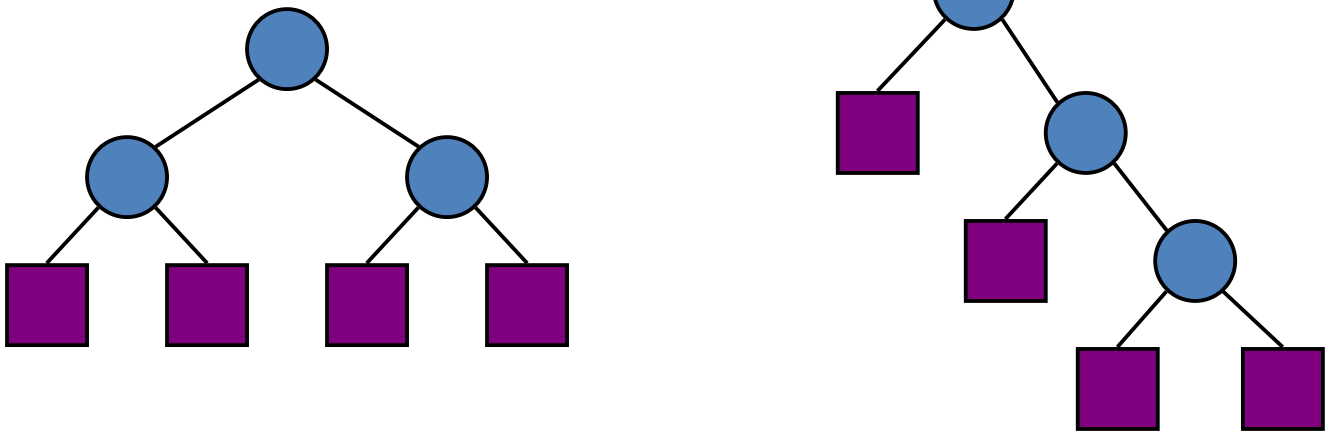
Decision Tree

- Binary tree associated with a decision process
  - internal nodes: questions with yes/no answer
  - external nodes: decisions
- Example: dining decision



Properties of Binary Trees

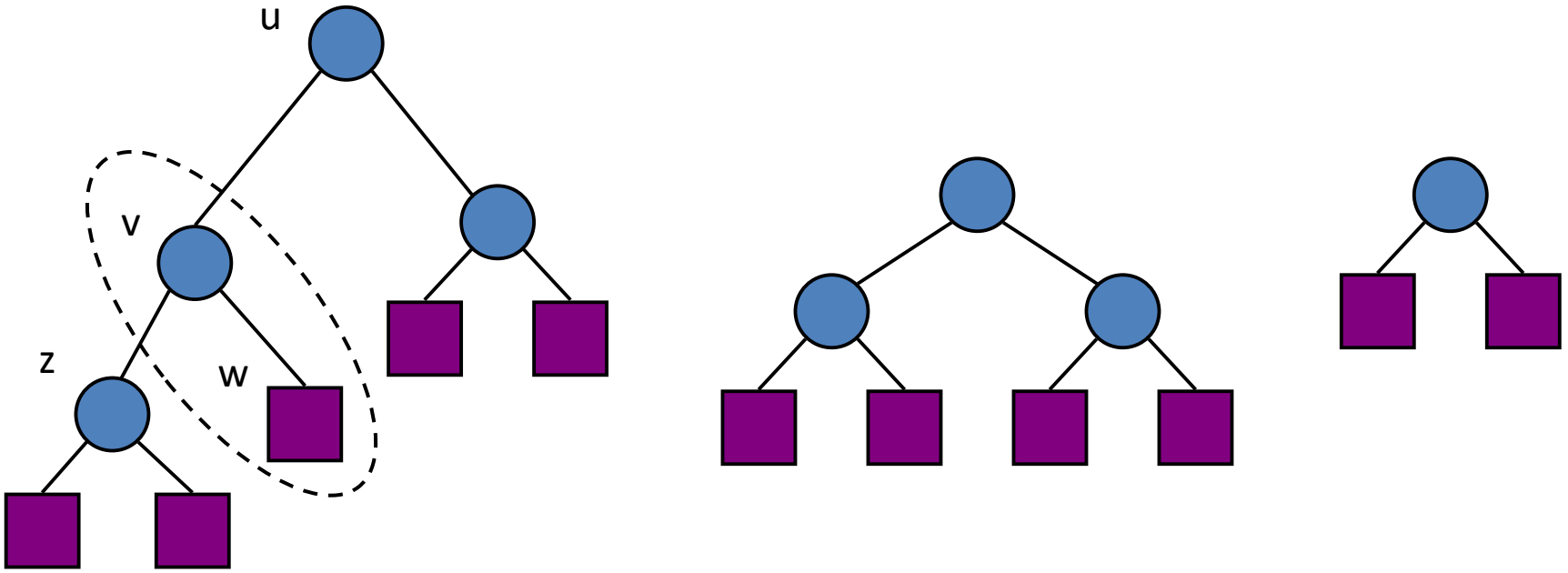
- Let T be a (proper) binary tree with n nodes, and let h denote the height of T. Then T has the following properties.
  - The number of external (leaf) nodes in T is at least h+1 and at most 2^h.
  - The number of internal nodes in T is at least h and at most 2^h-1.
  - The number of nodes in T is at least 2h+1 and at most $2^{h+1}-1$.
  - The height h of T satisfies $\log(n+1) \leq h \leq (n-1)/2$.
- Proof:



Properties of Binary Trees

- In a (proper) binary tree T, the number of external nodes is 1 more than the number of internal nodes.

- Proof:

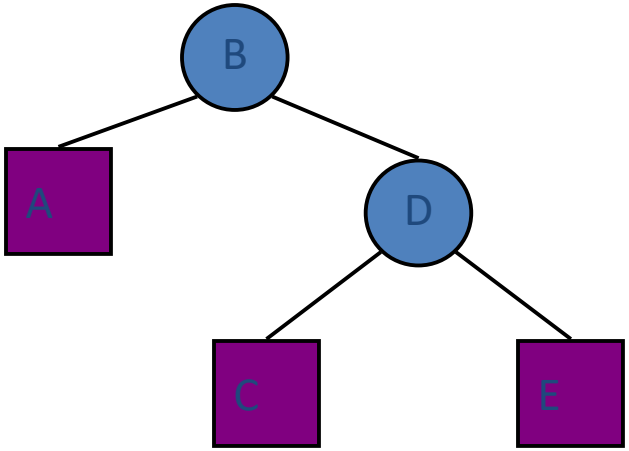


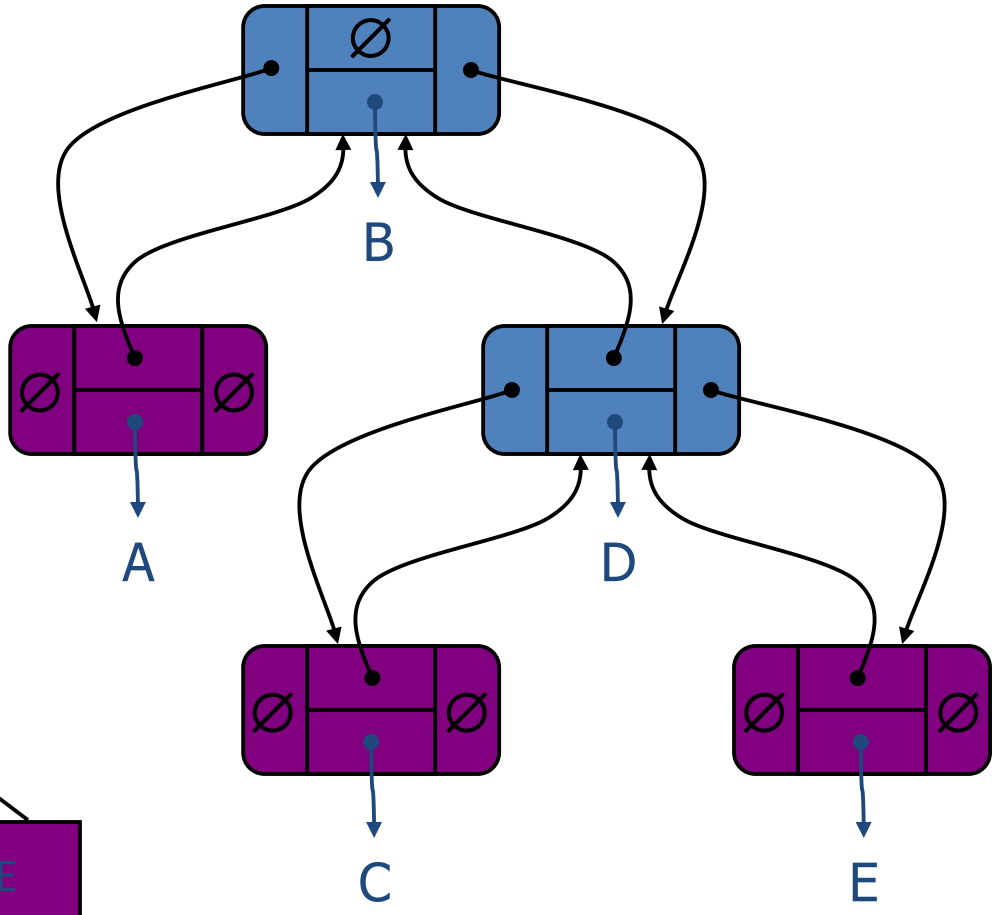
BinaryTree ADT

- The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- Additional methods:
  - position leftChild(v): returns the left child of v
  - position rightChild(v) : returns the right child of v
  - position sibling(v) : returns the sibling of node v

Linked Structure for Binary Trees

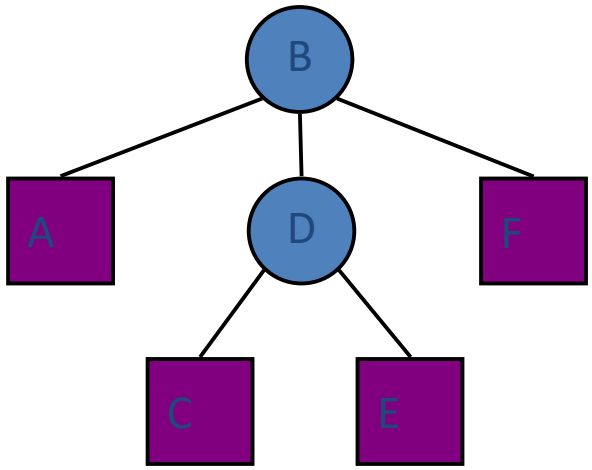
- A node is represented by an object storing
  - Element
  - Parent node
  - Left child node
  - Right child node

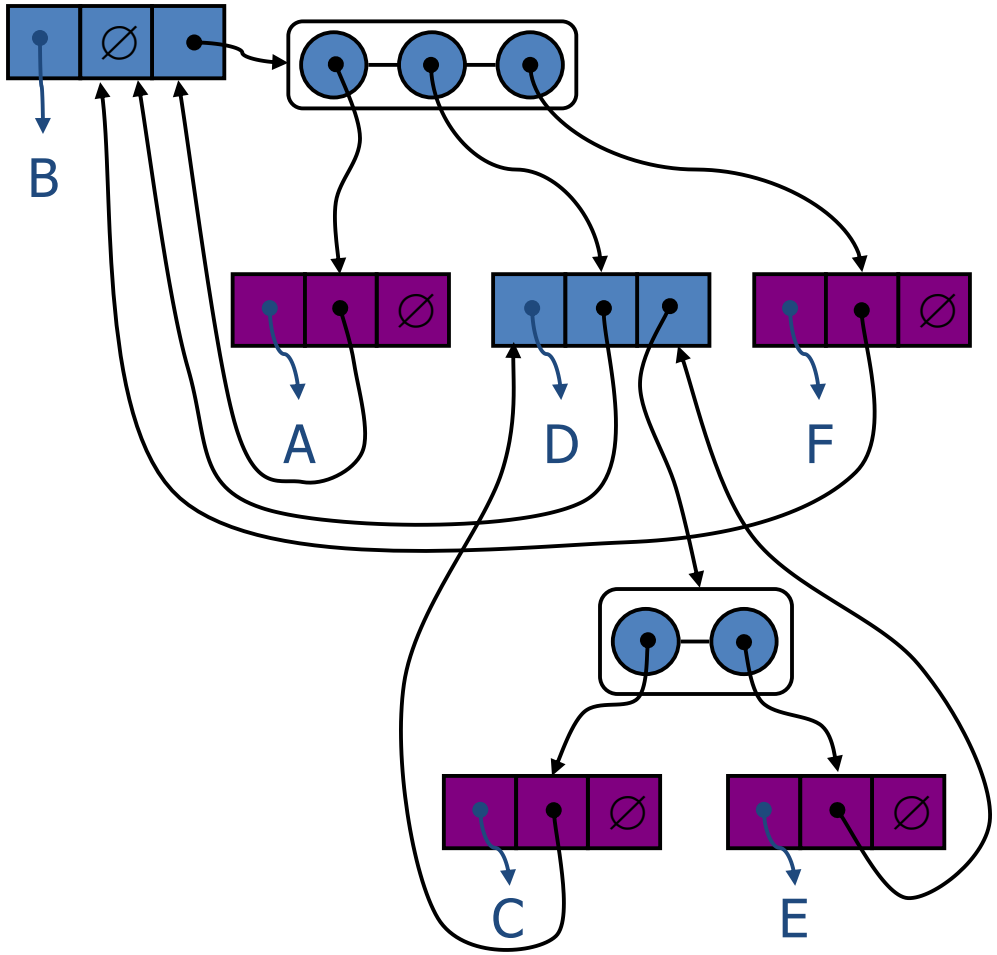




Linked Structure for General Trees

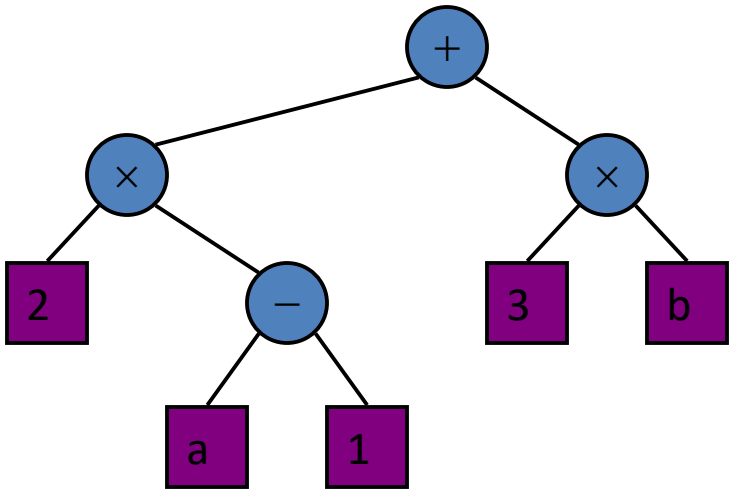
- A node is represented by an object storing
  - Element
  - Parent node
  - Children Container: Sequence of children nodes





Print Arithmetic Expressions

- Specialization of an inorder traversal
  - print operand or operator when visiting node
  - print "(" before traversing left subtree
  - print ")" after traversing right subtree

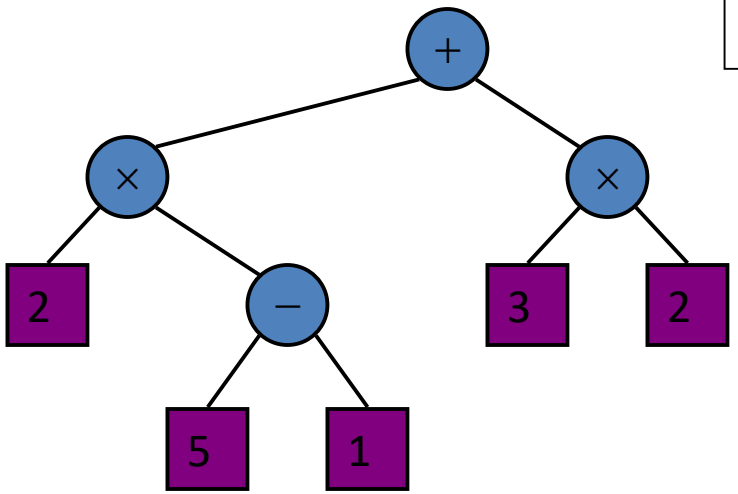


Algorithm *printExpression*(*v*)
  if *hasLeft* (*v*)
    print("(")
    inOrder (*left*(*v*))
  print(*v.element* ())
  if *hasRight* (*v*)
    inOrder (*right*(*v*))
    print (")")

((2 × (a – 1)) + (3 × b))

Evaluate Arithmetic Expressions

- Specialization of a postorder traversal
  - recursive method returning the value of a subtree
  - when visiting an internal node, combine the values of the subtrees



Algorithm *evalExpr*(*v*)
  if *isExternal* (*v*)
    return *v.element* ()
  else
    x ← *evalExpr*(*leftChild* (*v*))
    y ← *evalExpr*(*rightChild* (*v*))
    ◊ ← operator stored at *v*
    return *x* ◊ *y*